

PyCC.id: A Python package for nonlinear equation discovery based on characteristic curves

February 10, 2026

Summary

`PyCC.id` (or simply `PyCC`) is a user-friendly and highly-customizable Python package for data-driven equation discovery. It is designed to address the critical challenge of identifiability by incorporating domain-specific prior information into the modeling process. This is achieved through the use of user-defined structural families, which restricts the search to structurally identifiable forms. When the hypothesized structure is selected to satisfy certain uniqueness properties, the methodology guarantees identifiability, thereby allowing the practitioner to rigorously verify the validity of the proposed physical structure. Furthermore, by using characteristic curves (CCs) to define the constitutive relations of the system elements, the method ensures that the resulting models are inherently interpretable and physically consistent.

For systems described by ordinary differential equations (usually expressed as a set of first-order equations $d\mathbf{x}/dt = \mathbf{F}(\mathbf{x}, t)$, where $\mathbf{x}$ is the state vector and $\mathbf{F}$ is the vector field that defines the system), the CC-based framework allows the practitioner to decompose the vector field according to a physically motivated structure:

$$\frac{d\mathbf{x}}{dt} = \mathbf{G}(\mathbf{x}, \mathbf{F}_{ext}(t); \{\mathbf{f}\}, \mathbf{a})$$

Here, $\mathbf{G}$ represents a user-defined structural family that reflects the hypothesis of the practitioner about the system (e.g., *a second order system with velocity-dependent friction*). The inputs $\mathbf{x}$ and $\mathbf{F}_{ext}(t)$ represent state vector and external forces, respectively. The unknowns to be discovered are: (1) a set of one-dimensional functions $\{\mathbf{f}\} = \{f_1, f_2, \dots\}$, referred to as the CCs; and (2) a set of scalar parameters $\mathbf{a} = \{a_1, a_2, \dots\}$.

The package offers multiple backends for modeling these CCs using multiple training methodologies: neural networks (NN) via PyTorch [1], polynomial basis functions (Poly), and symbolic regression (SymbR) via PySR [2] (which can be used to discover analytical expressions directly or as a post-processing tool). Physical knowledge (such as geometric symmetries, boundary conditions, or conservation laws) can be incorporated as explicit constraints during training, guiding the discovery toward identifiable and physically consistent models.

Statement of Need

The Identifiability Challenge:

When reconstructing governing equations from empirical observations, practitioners face the ill-posed nature of the inverse problem: multiple distinct mathematical models can often yield similar prediction errors, making model selection ambiguous and failing to guarantee a unique representation of the true underlying mechanisms. This identifiability problem is particularly relevant in purely data-driven methods, where the lack of constraints allows algorithms to find spurious models that properly fit the data but fail to capture the actual physics. Existing approaches face significant limitations in addressing this challenge:

Opaque predictive models: Methods like Neural ODEs [3] treat the entire vector field $\mathbf{F}$ as a monolithic NN. Multiple distinct internal representations can produce similar predictions, leading to models that may be flexible and precise but fail to be consistent with the underlying physics [4]. This lack of uniqueness makes physical interpretation difficult or impossible.

Interpretable sparse and symbolic regression methods: Approaches based on sparse regression (e.g., SINDy [5]) and symbolic regression (e.g., PySR [2]) aim to find analytical equations from data. However, these methods often face similar identifiability challenges; depending on the candidate library or set of operations, fundamentally different system structures

may yield nearly identical error values. This leads to model non-uniqueness, where multiple non-equivalent equations appear equally valid.

Structured Discovery with Prior Knowledge:

One way to address these identifiability issues is by incorporating physical prior knowledge about the system. PyCC package addresses this by using the formalism of CCs, which enables practitioners to:

- i) Incorporate prior information systematically: Physical knowledge—including geometric symmetries (e.g., odd/even functions), forcing a specific value for the CCs (e.g., $f_1(2) = 1$), and conservation laws—can be embedded as explicit constraints. This drastically reduces the hypothesis space and acts as an inductive bias that guides the search away from spurious solutions.
- ii) Work within identifiable structural families: When a structural hypothesis is properly specified (e.g., by selecting a second-order oscillator family), the framework can guarantee unique decompositions of the system dynamics, enabling rigorous hypothesis testing and systematic model validation.
- iii) Achieve physical consistency: Each discovered CC corresponds to an independent physical component (e.g., a specific spring or damper) rather than an abstract mathematical artifact. For example, if a practitioner hypothesizes a second-order system with a velocity-dependent friction, the framework inherently excludes spurious cross-terms that lack a physical basis.
- iv) Maintain interpretability: Since each CC is a univariate function, it can be directly visualized through simple plotting. Even when employing NNs, the restriction to one-dimensional functions maintains transparency: a NN with a single input and output is simply a curve that can be inspected to understand the underlying physics. This allows for straightforward visual verification and validation of the identified models.

The CC-based framework was initially developed for first-order dynamical systems using Fourier [6] and polynomial [7] representations. While these early analytical formulations established the general modeling paradigm, they were inherently restricted to first-order systems. Subsequent research extended the CC-based methodology to second-order systems [8] using more flexible representations, specifically polynomial basis functions (Poly-CC), sparse regression (SINDy-CC), and neural networks (NN-CC).

Validation of this generalized approach has been performed on representative benchmarks, including chaotic Duffing oscillators and discontinuous stick-slip systems. While initial studies focused on idealized noiseless conditions, recent work has demonstrated robust performance under varying Gaussian noise levels, showing how the incorporation of structural hypotheses systematically improves discovery [9]. This latest research also formalizes the extension to higher-order dynamics, emphasizing a key methodological advantage: by defining a family of admissible physical models, the framework ensures identifiability by design while retaining the use of universal approximators such as NNs to learn specific constitutive relations. This strategy effectively transforms the discovery process into an exploratory search across hypothetical model families, facilitating a thorough assessment of consistency, interpretability, and physical consistency.

Relation to other packages

- **Sparse identification (e.g., SINDy):** SINDy [5] is highly effective when the true system dynamics can be represented as a sparse combination of terms from a pre-defined library of candidate functions. However, its success relies on the user correctly defining the necessary basis functions. If the governing physical law involves a complex or non-standard term (e.g., a specific friction model or a saturation curve) that is absent from the library, SINDy may fail to identify the model or return a misleading approximation. In contrast, PyCC focuses on reconstructing the shape of the unknown CCs directly. This allows the use of universal approximators, such as NNs, to capture arbitrary functional forms without a rigid prior library. This CC-based methodology possesses intrinsic physical meaning and can be subsequently interpreted using post-hoc processing tools. For instance, the CCs can be fitted with symbolic or sparse regression tools to obtain analytical expressions.
- **Symbolic Regression (e.g., PySR):** Standard symbolic regression packages, such as PySR [2], typically perform an unconstrained search for a single equation that best fits a multivariate dataset. PyCC integrates PySR into the CC-based formalism, offering two distinct workflows:
 - Iterative symbolic discovery (SymbR method): PyCC frames the discovery process as an iterative optimization loop. Instead of searching for the full equation at once, it updates the individual f_i functions sequentially. In each step of the loop, a single-variable symbolic regression is performed to fit one f_i against the residual of the user-defined equation. This effectively breaks the complex, multivariate discovery problem into a sequence

of simpler, one-dimensional symbolic fitting tasks that collectively minimize the global error of the proposed structure (see details in Ref. [9]).

- Symbolic post-processing previously obtained CCs: This is a two-step workflow, where the practitioner first identifies the model using a method other than **SymbR** (e.g., using the NN method). Once the shapes of the CCs are obtained, **PySR** is applied to find the analytical forms for each one-dimensional curve individually. This approach avoids the difficulty of applying symbolic regression directly to the raw data (as the **SymbR** method does). Instead, it targets much simpler, cleaner one-dimensional curves, a strategy that has demonstrated superior robustness to noise compared to the **SymbR** iterative approach[9].
- **Neural ODEs:** Standard Neural ODE packages [3] typically learn the entire vector field $\mathbf{F}(\mathbf{x}, t)$ using a single, monolithic NN. While highly flexible and often powerful for pure prediction, these models function as *black boxes* that offer little insight into the underlying physics. Instead, **PyCC** adopts a *grey-box* philosophy: it models only the specific, unknown constitutive relations $\{\mathbf{f}\}$ within a user-defined physical structure $\mathbf{G}$. Regardless of the backend used to model these curves (e.g., NNs, polynomials, or symbolic regression), the overarching structure ensures that the learned components retain a clear physical interpretation (e.g., distinct *damping functions* or *restoring forces*).
- **Physics-Informed Neural Networks:** PINNs[10] represent a powerful paradigm for solving forward and inverse problems, embedding physical laws directly into the loss function. While excellent for solving differential equations or estimating parameters within known or partially known governing equations, standard PINNs are typically less suited for equation discovery when the functional forms of the interactions are entirely unknown. **PyCC** differs by explicitly isolating these unknown terms as distinct functions to be learned, making them directly accessible for independent analysis, visualization, and physical interpretation.

PyCC library is not intended to compete with these established packages, but rather to serve as an integrative tool that integrates their strengths (such as the symbolic power of **PySR** or the flexibility of NNs) within a unique, structured framework that prioritizes identifiability and physical consistency.

Core Functions and Methods

This section briefly describe the three main methods that are defined in the **PyCC** library: `pycc.simulate()`, `pycc.train()`, and `pycc.post_processing()`.

`pycc.simulate()`

This function is responsible for integrating forward system dynamics over time. It acts as a simulation manager, dispatching the task to specific strategies (such as “Theoretical”, “NN”, “SymbR”, “Poly”, or “Interp”) based on the selected method.

Main Uses

- Theoretical Simulation: Can be used to integrate forward the theoretical equations in order to obtain ground-truth datasets that will be used for training the models, or to generate theoretical forward integration under different initial conditions or driven forces to compare against model predictions.
- Validation: Integrates discovered models after training (e.g., from a NN model) to verify their accuracy against expected dynamics.

`pycc.train()`

This function serves as the primary entry point for identifying system dynamics from a given dataset. It operates as a training dispatcher that abstracts the complexity of individual algorithms, automatically routing data to the appropriate module based on the specified “method” parameter.

Main uses

- System Identification: seamless switching between different identification techniques (such as Neural Networks, Polynomial Regression, or Symbolic Regression) without the need to restructure data or equation definitions.

`pycc.post_processing()`

This acts as a standalone utility, specifically designed to obtain analytical expressions for the obtained CCs. It converts numerical CCs (previously obtained from methods such as ‘NN’ or ‘Poly’) into explicit symbolic expressions.

Main Uses

- Model Conversion: Transforms obtained CCs into symbolic functions for inspection, analysis, and storage.
- Pre-Simulation Preparation: Generates clean symbolic models and plots to validate fit quality before they are fed back into `pycc.simulate()`.

Illustrative Example: A Second-Order System

To demonstrate the capabilities of ‘PyCC’, we consider a nonlinear oscillator with friction. The governing equation is a second-order differential equation:

To demonstrate the capabilities of `pyCC`, we consider a classic physical system: a nonlinear oscillator with friction. The governing equation is a second-order differential equation:

$$\ddot{x} + \delta \dot{x} + \mu \tanh(500\dot{x}) + \alpha x + \beta x^3 = F_{ext}(t)$$

where $F_{ext}(t) = A \cos(\omega t)$ is an external driving force. The term $\tanh(500\dot{x})$ acts as a smooth approximation of the signum function, $\text{sign}(\dot{x})$, effectively modeling Coulomb friction.

To apply the `pyCC` framework, we rewrite this system as a set of first-order ordinary differential equations (ODEs). By defining the state variables $x_1 = x$ (position) and $x_2 = \dot{x}$ (velocity), the system becomes:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - \delta x_2 - \mu \tanh(500x_2) - \alpha x_1 - \beta x_1^3 \end{cases}$$

After simulating this system to generate synthetic data, the resulting dataset $\mathcal{D} = \{x_1, x_2, \dot{x}_1, \dot{x}_2, F_{ext}\}$ serves as the input for the system identification task.

The following code shows how to use the PyCC library to integrate the theoretical equations in order to generate the input dataset that will be used later for training the models.

```
import pycc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#####
# Simulating a stick-slip second order system using
# pycc.simulate(method="Theoretical")
# Defining parameters and theoretical functions
alpha=1.0;beta=0.2;delta=0.1;Omega=1.0;
x0=0.0;v0=0.0; y0=[x0,v0] # initial conditions
t_span=(0, 20); t_eval=np.linspace(*t_span, 1000)
def F1_th(x_dot):
    return delta * x_dot + 0.5 * np.tanh(500*x_dot)
def F2_th(x):
    return alpha * x + beta * x**3
def F_ext(t):
    return np.cos(Omega * t)
# Defining the theoretical equation
eqs_th = ['x1_dot = x2',
          'x2_dot = F_ext - f1(x2) - f2(x1)']
# Defining the simulation parameters
params_th = {
    't_span': t_span,
    'y0': y0,
    't_eval': t_eval,
    'method': 'LSODA',
    'local_funcs': {'f1': lambda t: F1_th(t),\
                    'f2': lambda t: F2_th(t),'F_ext': lambda t: F_ext(t)}}
}
```

```

# Integrating forward the theoretical equation
sol,derivatives = pycc.simulate(eqs_th,method="Theoretical", params=params_th)
# Extracting data
time_data      = sol.t
x1_data        = sol.y[0]
x2_data        = sol.y[1]
x1_dot_data    = derivatives[0]
x2_dot_data    = derivatives[1]
F_ext_val      = F_ext(time_data)

```

Defining an Identification Strategy

With PyCC, you can tackle the identification problem using three distinct strategies, depending on how much prior knowledge you wish to incorporate:

(i) Functional Approach In this approach, we assume a *structural skeleton* for the physics but leave the specific constitutive relations as unknown functions to be learned from data. The practitioner hypothesizes that the system consists of a velocity-dependent damping force and a position-dependent restoring force:

$$\ddot{x} = F_{ext}(t) - f_1(\dot{x}) - f_2(x)$$

This family of second order systems is called as *velocity-dependent friction models with external force* (which is a generalized Rayleigh-type nonlinear oscillation with velocity-dependent friction and external forcing) and has uniqueness properties as discussed in Refs. [8] and [9]. Translating this to the state-space representation required by PyCC:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - f_1(x_2) - f_2(x_1) \end{cases}$$

The Goal: Discover the shapes of the CCs $f_1(x_2)$ and $f_2(x_1)$. These functions can be approximated using Neural Networks (NN-CC), symbolic regression (SymbR-CC), or polynomial expansions (Poly-CC).

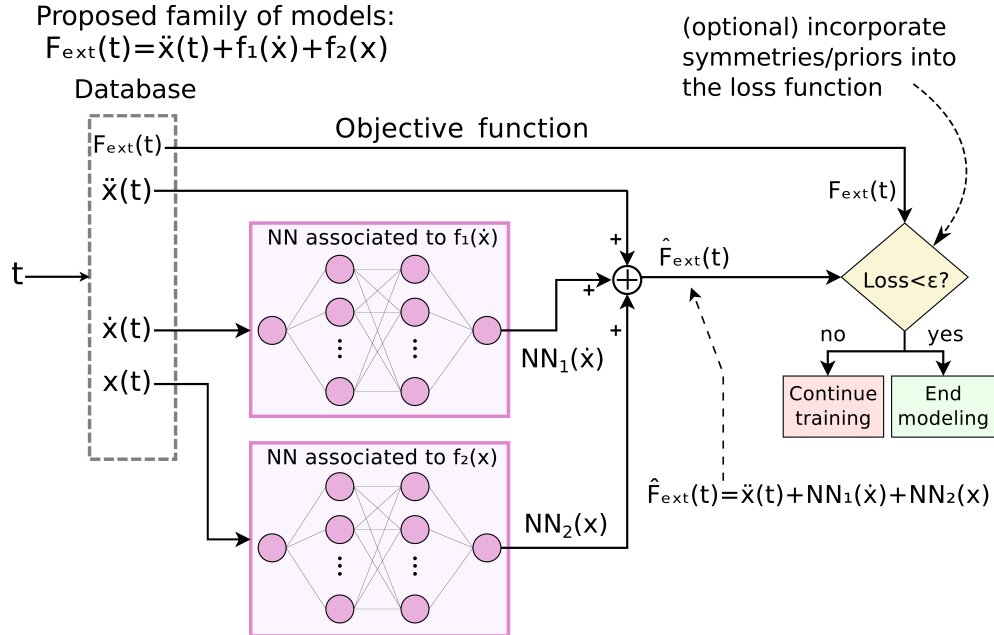


Figure 1: The architecture for a second-order system. Two independent neural networks (NN_1 and NN_2) approximate the unknown CCs. NN_1 sees only velocity, and NN_2 sees only position. Adapted from Ref. [9]

Why this matters: This architecture enforces *uniqueness and physical consistency* (see Refs. [8] and [9]).

The following script demonstrates the complete identification workflow. It begins by constructing the training database from previously generated variables and defining the governing equations using the functional approach. Next, it trains a model using the NN method and performs forward simulations to validate the discovered dynamics against theoretical solution.

```
#####
# Defining the database for training
df = pd.DataFrame({
    'x1':x1_data,
    'x2':x2_data,
    'x1_dot':x1_dot_data,
    'x2_dot':x2_dot_data,
    'F_ext': F_ext_val
})
# Defining the proposed equations to use for identification
eqs = [
    'x1_dot = x2',
    'x2_dot = F_ext - f1(x2) - f2(x1)'
]
# In this example, we use the functional approach
# with the NN-CC method [pycc.train(method='NN')]
# thus, we must define the
# defining constraints for training (optional)
constraints = [ # adding prior known information
    {'constraint': 'f2(0)=0'},
    {'constraint': 'f1 odd'},
    {'constraint': 'f2 odd'},
]
# defining training parameters (optional)
params_NN = {
    'neurons': 100,
    'layers':3,
    'lr': 1e-4,
    'epochs': 2000,
    'error_threshold': 1e-6,
    'extrapolation': None,
    'device':'cpu',
    'weight_loss_param': 1e-3,
    'constraints': constraints,
}
# training the model using NN-CC method
models, evals, obtained_coefs = pycc.train(df, eqs,method='NN', params=params_NN)
# plotting obtained functions f1 and f2
x_f1_cc, f1_cc, x_f2_cc, f2_cc = evals
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].plot(x_f1_cc, f1_cc, label='$f_1$ learned NN-CC')
ax[0].plot(x_f1_cc, F1_th(x_f1_cc), '--', label="$f_1$ theory")
ax[0].set_xlabel('$x_2$')
ax[0].set_ylabel('$f_1(x_2)$')
ax[0].legend()
ax[1].plot(x_f2_cc, f2_cc, label='$f_2$ learned NN-CC')
ax[1].plot(x_f2_cc, F2_th(x_f2_cc), '--', label="$f_2$ theory")
ax[1].set_xlabel('$x_1$')
ax[1].set_ylabel('$f_2(x_1)$')
ax[1].legend()
plt.tight_layout()
plt.show()
# Print learned parameters (if any)
if obtained_coefs:
```

```

print("\nLearned scalar parameters:")
for name, val in obtained_coefs.items():
    print(f"{name} = {val.item():.4f}")

#####
# simulating forward the identified model
# using NN-CC method [pycc.simulate(method='NN')]
# defining simulation parameters
params_NN_simul = {
    'models': models,
    'obtained_coefs': obtained_coefs,
    'local_funcs': {'F_ext': lambda t: F_ext(t)},
    't_span': t_span,
    'y0': y0,
    't_eval': t_eval,
    'method': 'LSODA', # solve_ivp
    'atol': 1e-8,
    'rtol': 1e-6,
    'check_nan': True
}
# integrating identified equations
sol,_ = pycc.simulate(eqs, method='NN', params=params_NN_simul)
time_sim=sol.t
x1_sim=sol.y[0]
x2_sim=sol.y[1]
# Plotting identified vs theoretical solution
plt.figure()
plt.plot(time_sim, x1_sim, label="x(t) simulated with NN method")
plt.plot(time_data, x1_data, label="x(t) th")
plt.xlabel('t')
plt.ylabel('x(t)')
plt.legend()
plt.show()

```

Alternatively, the following script demonstrates a post-processing workflow where the CCs (previously obtained via methods such as NN), are fitted analytically and then simulated forward using `pchip` interpolation. While it is possible to simulate the obtained system using the analytical expressions directly, the interpolation method used here is significantly faster. Provided the sampling density (`n_eval`) is sufficiently large, the discrepancy between the interpolated model and the analytical expression is negligible.

```

#####
# Post-processing the identified CCs with
# SymbR [pycc.post_processing()].
print("post-SR processing for the CCs")
# Defining settings for the PySR fit
pysr_settings = {
    'niterations': 100,
    'populations': 20,
    'binary_operators': ['+', '*', '-'],
    'unary_operators': ['tanh', 'sin', 'cos'],
    'maxsize': 20
}
# Defining the 'params' dictionary for the post-processing function
post_process_params = {
    'evals': evals,
    'pysr': pysr_settings,
    'plot': True, # This will show the plots of the fits
    'n_eval': 200, # Generate 200 points for the new evals_sr
}
# Running the post-processing

```

```

# This command also prints the fits and shows the plots
models_sr,evals_sr = pycc.post_processing(eqs, method='SymbR', params=post_process_params)

#####
# We can use the post-processed CCs for forward simulation
# Simulating the post-SymbR models using interpolation method
params_sim_interp = {
    'evals': evals_sr,          # Use the NN's characteristic curves
    'obtained_coefs': obtained_coefs, # Use the scalars from the NN fit
    'local_funcs': {'F_ext': F_ext},
    'interp_method': 'pchip',    # Use shape-preserving cubic
    't_span': t_span,
    'y0': y0,
    't_eval': t_eval,
}
# We simulate the system by interpolating the CCs (defined with the 'evals_nn' data variable).
sol, derivs = pycc.simulate(eqs, method='Interp', params=params_sim_interp)
print("Integration success:", sol.success)
time_sim=sol.t
x1_sim=sol.y[0]
x2_sim=sol.y[1]
# Identified vs theoretical solution
plt.figure()
plt.plot(time_sim, x1_sim, label="x(t) simulated NN-CC(+sym+post-SR)")
plt.plot(time_data, x1_data, label="x(t) th")
plt.xlabel('t')
plt.ylabel('x(t)')
plt.legend()
plt.show()

```

(ii) Parametric Approach If the practitioner has a strong hypothesis regarding the specific functional forms (e.g., “I know it is a Duffing oscillator with Coulomb friction”), PyCC can be used to identify the unknown parameters directly. The system equations are defined with explicit parameters:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - a_1 x_2 - a_2 \tanh(a_3 x_2) - a_4 x_1 - a_5 x_1^3 \end{cases}$$

The Goal: Find the optimal values for the scalar parameters $\{a_i\}$ using nonlinear optimization. This effectively acts as a robust parameter estimation framework.

The only difference in this code to previous scripts is the equations definition used for model training:

```

# Defining the proposed equations to use for identification
eqs = [
    'x1_dot = x2',
    'x2_dot = F_ext - a1 x2 - a2 tanh(a3 x2) - a4 x1 - a5 x1^3'
]

```

(iii) Hybrid Approach The library also enables a hybrid approach, combining functional and parametric methods. Practitioners can prescribe known functional forms for specific terms (anchoring the model in established physical laws) while leaving other components as unknown functions.

For example, if the restoring force is known to be a cubic spring, but the friction model is unknown, one might define:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - f_1(x_2) - a_4 x_1 - a_5 x_1^3 \end{cases}$$

The Goal: Simultaneously identify the unknown function $f_1(\dot{x})$ (the friction law) and the scalar stiffness parameters a_4 and a_5 .

The only difference in this code to previous scripts is the equations definition used for model training:

```
# Defining the proposed equations to use for identification
eqs = [
    'x1_dot = x2',
    'x2_dot = F_ext - f1(x2) - a4 x1 - a5 x1^3'
]
```

PyCC reserves the names f_i and a_i for functions and scalar parameters, respectively, where $i = \{0, 1, \dots\}$.

Implementation Details The following code presents the implementation details using the PyCC library:

Features

PyCC is designed to be a user-friendly and highly-customizable tool for researchers and practitioners. Its key features include:

- **Interpretable Models:** It allows practitioners to add prior information by decomposing the complex, high-dimensional functional space into a set of simple, one-dimensional CCs that have direct physical meaning (i.e., the CCs are directly the constitutive relations of the system elements, such as, stiffness or damping), and a set of scalar parameters (e.g., mass).
- **Flexible Function Parametrization:** Supports multiple backends for modeling the CCs, allowing practitioners to switch between different representations with minimal modifications:
 - **Neural Networks (NN):** Implemented using PyTorch [1] and compatible with multicore CPUs and GPUs.
 - **Polynomials (Poly):** Provides an expansion of the f_i functions using polynomial basis functions.
 - **Symbolic Regression (SymbR):** Uses PySR package [2] to discover analytical expressions that are compatible with the given prior structure and also as a post-processing tool.
- **Physics-Informed Discovery:** Allows users to inject domain knowledge as constraints during training (e.g., 'f1 odd', 'f2(0)=0') or by defining conserved quantities that are added to the loss function. This leads to more robust and physically consistent models.
- **Hardware Acceleration:** Natively supports multicore CPUs and GPUs from both NVIDIA (cuda) and Intel (xpu via intel-extension-for-pytorch) for training the NNs.
- **Built-in Simulator:** Includes a versatile ODE solver (pycc.simulate) compatible with all identification methods. This solver facilitates forward integrations of both the identified models and the theoretical equations used to generate training datasets.
- **Comprehensive Documentation:** Provides a Google Colab notebook for a quick start, as well as complete documentation and a gallery of tutorials and examples in the corresponding repository.

Mentions of scholarly publications

The pyCC package provides a generalized software implementation of the CC-based approaches for system identification. This core methodology was central to methods first introduced for first-order systems [6, 7] and later extended to second-order systems [8, 9]. PyCC unifies and extends this approach into a single framework capable of handling higher-order dynamical systems.

Key References

The software package is available at: <https://github.com/FedejGon/pyCC.id>

Acknowledgements

This work was partially supported by CONICET (Consejo Nacional de Investigaciones Científicas y Técnicas, Argentina). We acknowledge the computational resources from the Clementina XXI supercomputer and CCT-Rosario Computational Center, both managed by the High Performance Computing National System (SNCAD, ME-Argentina), with the support of the Undersecretariat of Science and Technology of Argentina.

References

- [1] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [2] M. Cranmer. “Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl”. In: *arXiv preprint arXiv:2305.01582* (2023). eprint: 2305.01582. URL: <https://arxiv.org/abs/2305.01582>.
- [3] R.T.Q. Chen et al. “Neural Ordinary Differential Equations”. In: *NeurIPS 2018*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018.
- [4] K. Wu. “Automations Black-Box Conundrum: The Interpretability Crisis of Data-Driven System Identification and the Path Forward”. In: *Theoretical and Natural Science* 120.1 (July 2025), pp. 25–35. ISSN: 2753-8826. DOI: 10.54254/2753-8818/2025.ad25063. URL: <http://dx.doi.org/10.54254/2753-8818/2025.AD25063>.
- [5] S. L. Brunton, J. L. Proctor, and J. N. Kutz. “Discovering governing equations from data by sparse identification of nonlinear dynamical systems”. In: *Proceedings of the National Academy of Sciences* 113.15 (2016), pp. 3932–3937. DOI: 10.1073/pnas.1517384113.
- [6] F. J. Gonzalez. “Determination of the characteristic curves of a nonlinear first order system from Fourier analysis”. In: *Sci. Rep.* 13.1 (Feb. 2023), p. 1955. DOI: 10.1038/s41598-023-29151-5.
- [7] F. J. Gonzalez. “System identification based on characteristic curves: a mathematical connection between power series and Fourier analysis for first-order nonlinear systems”. In: *Nonlinear Dyn.* 112.18 (July 2024), pp. 16167–16197. ISSN: 1573-269X. DOI: 10.1007/s11071-024-09890-4.
- [8] F. J. Gonzalez and L. P. Lara. “Interpretable neural network system identification method for two families of second-order systems based on characteristic curves”. In: *Nonlinear Dyn.* 113.24 (Sept. 2025), pp. 33063–33086. ISSN: 1573-269X. DOI: 10.1007/s11071-025-11744-6.
- [9] F. J. Gonzalez. *Integrating prior knowledge in equation discovery: Interpretable symmetry-informed neural networks and symbolic regression via characteristic curves*. 2026. arXiv: 2601.21720 [nlin.CD]. URL: <https://arxiv.org/abs/2601.21720>.
- [10] M. Raissi, P. Perdikaris, and G.E. Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational Physics* 378 (2019), pp. 686–707. ISSN: 0021-9991. DOI: 10.1016/j.jcp.2018.10.045.